Phase 1 validation: golden-master extraction

First deliverable of Phase 1 (streaming state machine). Before writing any streaming code, the locked FVG model was instrumented to emit a complete event-by-event trail -- not just the final 603 trades -- so the streaming state machine can be diffed against it stage-by-stage, per the project's standing rule that every change gets tested in isolation before being trusted.

What was done

- `phase1/convert_mt_to_ascii.py` and `phase1/extract_golden_master.py` written. See `phase1/README.md` for what each does and how to regenerate the output.
- Constraint on `extract_golden_master.py`: it must not change `FVG_model.py`'s trading behavior in any way -- only add logging. Verified this by structurally diffing the instrumented file against the original after stripping all `log_event(...)` calls; the only remaining differences were the docstring header, a few dropped inline comments, two semicolon-joined statements split onto separate lines, and one variable rename confined to the post-loop reporting section (`n` -> `n_trades`, after trading logic has already finished). No control flow, condition, or calculation was touched.

Bug found before running anything

Data format mismatch would have silently used ~1.5 years of data instead of 10.5. The uploaded 2016-2023 files were HistData's "MT" format (comma-separated, split date/time columns); 2024-2026 were "ASCII" format (semicolon-separated, combined timestamp). `FVG_model.py`'s loader globs only the ASCII filename pattern (`DAT_ASCII_EURUSD_M1_*.csv`) -- running it as-is against a directory containing both formats would have matched only the 2024-2026 files, with no error, no warning, and a "successful"-looking but badly wrong run.

Fixed by writing `convert_mt_to_ascii.py` rather than modifying the loader itself -- converts MT-format files into the same on-disk shape the ASCII files are already in, so the existing (already-verified) loader code in `FVG_model.py` doesn't need to change at all.

Verified before trusting the conversion:

- Row count in == row count out, per file, no silent row loss.
- First/last row of each converted file spot-checked.
- Day-boundary continuity checked across the format transition (both conventions roll over at 17:00 NY time, consistent with the same underlying HistData timestamp convention).

One data-quality observation, not a bug: 2023 has meaningfully fewer rows per day (~1037 avg) than 2022 (~1195 avg) despite almost the same number of trading days. Rows-in

matched rows-out exactly for every file, so this isn't a conversion artifact -- likely genuine gaps in quiet/ illiquid source minutes. Did not end up affecting the result (see below).

Result

Ran against the full, real 2016-2026 dataset (not a subset, not synthetic data):

Metric	Extracted	Locked (PROJECT_DESCRIPTION.md)	Match
1-min bars	3,820,900	~3.82 million	Yes
5-min bars	767,151	~767,000	Yes
Trades	603	603	Yes
Win rate	66.2%	66.2%	Yes
Return	+397.4%	+397.4%	Yes
Wins/Losses/Scratches	389/199/15	389/199/15	Yes

Year-by-year trade counts (the full table, not just the headline numbers) matched `PROJECT_DESCRIPTION.md` exactly for every single year, 2016 through 2026 -- including 2023, despite the row-density observation above.

Output

- `golden_master_trades.jsonl` (committed, 120KB) -- the 603 trades in an easier-to-diff format than the original pickle.
- `golden_master_events.jsonl` (not committed, ~12MB -- see `phase1/README.md` for regeneration) -- 93,791 events across 9 types:

Event type	Count
intraday_swing_high_confirmed	31,929
intraday_swing_low_confirmed	31,735
raid_detected	14,727
mss_confirmed	6,382
fvg_found	2,138
day_trend_determined	1,951
day_skipped_no_trend	1,215
fvg_rejected_min_stop	1,201
order_filled	603
trade_closed	603
daily_swing_high_confirmed	452
daily_swing_low_confirmed	441
day_skipped_insufficient_bars	331
day_skipped_fomc	83

Note: 84 FOMC dates are defined in the script's date set, but only 83 `day_skipped_fomc` events fire -- expected, since the day-skip check only runs for days that actually appear in the resampled data (one FOMC date apparently doesn't land on a day present in `all_days`, not investigated further since it doesn't affect the trade-count match above).

Phase 1 validation, part 2: the streaming state machine

Built stage by stage after the golden-master extraction above, one component per stage, each independently validated against the golden master before the next was started. All validations below were run against the full, real 2016-2026 dataset.

Component-by-component results

Component	What it does	Exact-match result vs. golden master
`DailySwingDetector`	Daily swing highs/lows feeding the trend filter; centered 5-day window with explicit confirmation delay	**893/893** events -- every timestamp, day index, and price identical
`IntradaySwingDetector`	Same mechanism on 5-min bars; resets fresh each day via `start_new_day()`	**63,664/63,664** events (31,929 highs + 31,735 lows) identical
`RaidDetector`	Combines trend + confirmed swings to detect Kill Zone liquidity raids; check-then-update ordering reproduces the batch's `bisect_left` causality cutoff (proof in `phase1/streaming/README.md`)	**100% of 14,727** raids found, zero misses; 2,312 extras, every one individually verified to fall after a completed trade that day (the batch's early-exit, resolved later by `DayOrchestrator`)
`MSSWatch`	Short-lived per-raid watcher for market structure shift within a ~9-bar window	**100% of 6,382** MSS events found, zero misses; all extras traced to the two known early-exit gaps, 1,074 of them individually verified
`FVGDetector`	3-candle fair value gap check; rolling buffer, no confirmation delay	**100% of 2,138** FVG events found, zero misses; all 520 extras traced to the same two known gaps -- no new gap introduced
`TradeAttempt`	Min-stop rejection, limit fill, dynamic candle-midpoint target, win/loss/scratch tracking	Validated as part of the full-pipeline run below
`DayOrchestrator`	Coordinates everything per day; parallel candidate searches, lexicographic priority resolution	**The decisive result below**

Each component also has hand-constructed unit tests (in `tests/`) pinning its boundary behavior -- confirmation delays, tie handling, day-reset isolation, causality ordering, window expiry, same-bar fill+close, permanent abandonment, and the short-direction mirrors.

The bug this process caught -- the reason Phase 1 was built this way

The first attempt at wiring the five detection components together produced **318 trades instead of 603**, with 29 field mismatches among the trades it did find. All five components had individually perfect validations -- the bugs were entirely in the new wiring:

1. **Wrong scheduling model.** The wiring assumed "one raid's search at a time": once a raid began its MSS search, no new raid was considered until it resolved. The batch model instead treats every Kill Zone bar as a fresh raid candidate with its own full, independent search; a raid whose search fails entirely simply yields to the next bar's candidacy. Nearly half the golden master's trades come from exactly that situation. Diagnosed from the first missing trade date (2016-01-20: golden master shows a bar-24 raid whose search failed, then a bar-25 raid that won the day -- the broken wiring never gave bar 25 its turn).
2. **Target-window seed off by one bar.** `TradeAttempt` seeds were built from bars up to the MSS bar minus one, but the batch's target window (`highs[p-6:p]`) includes the MSS bar itself when the fill comes soon after it. This produced the 29 field mismatches (wrong targets on quickly-filled trades).

The fix was a real component, `DayOrchestrator`, not a patch to the validation script: every raid spawns a parallel candidate, every qualifying MSS bar spawns a parallel attempt, and the day's trade is the attempt with the lexicographically smallest (raid_bar, mss_bar) key among those that actually filled -- not the earliest fill in wall-clock time. Full statement of the rule and its rationale in `phase1/streaming/README.md`. This design also resolves the two "early exit" gaps documented for `RaidDetector` and `MSSWatch`: their extra events simply lose the priority selection instead of becoming trades.

Final result

The complete streaming pipeline -- daily trend, intraday swings, raids, MSS, FVG, trade attempts, day orchestration -- run bar by bar over the full 10.5-year dataset with no lookahead:

The streaming state machine reproduces the locked batch model exactly. This was the defining requirement of Phase 1 before any broker connectivity work (Phase 2) begins.

Still outstanding (optional hardening, not blockers)

- Randomized-slice differential testing (running both implementations over many random sub-periods) -- extra assurance beyond the full- history match, from the original mitigation list.
- The day-selection gates (FOMC / trend / minimum-bar-count / session windows) currently live in the validation script, not yet as a reusable component -- they'll need to become one when Phase 2 wires the orchestrator to a live data feed.
- Performance profiling of the per-bar path -- correctness came first; the parallel-candidate design has not yet been measured for live latency (almost certainly a non-issue at one bar per 5 minutes, but unmeasured is unmeasured).